

Cheat Sheet - Amazon Redshift

Course: HEIG-VD IST 2025/26

Authors: Victor Giordani · Khady Yade Diouf · Mouhamed Sakho · Abram Zweifel

Date: 12 juin 2026

Pourquoi Redshift ? (Cas d'usage HelvetiCart)

Scénario PME e-commerce suisse de 80 personnes. PostgreSQL de prod saturé par les requêtes analytiques : dashboards lents (agrégations sur 45 M de commandes), production menacée par les analyses ad-hoc, ~600 GB de données non centralisées (commandes, clickstream, exports marketing).

Ce que Redshift résout : décharger l'analytique lourde de la base transactionnelle vers un moteur dédié, sans migrer les données elles restent sur S3.

Inputs : données tabulaires depuis S3, Aurora (zero-ETL), RDS, Kinesis, ou toute source via JDBC/ODBC.

Outputs : résultats SQL consommés par des outils BI (QuickSight, Tableau, Looker), des APIs Data ou directement par des data scientists.

Bénéfices et limites

Bénéfices	Limites
Stockage colonnaire: scans 10-100x plus rapides que row-store	Latence plancher ~100 ms même sur <code>SELECT 1</code> et pas pour l'OLTP
MPP (Massively Parallel Processing): Parallélisation automatique des requêtes	INSERTs ligne à ligne : ~minutes pour 1 000 lignes
SQL compatible PostgreSQL (psql, BI, JDBC/ODBC)	DISTKEY / SORTKEY / VACUUM encore nécessaires en mode provisionné
Serverless : zéro admin, facturation à l'usage	Concurrence limitée sans <i>Concurrency Scaling</i> (payant)
Intégration native S3, IAM, Glue, QuickSight, Aurora	Pas de plafond de coût par défaut: Configurer usage limits J1
Result cache: Re-run instantané pour les dashboards	« 5x moins cher » : Vrai seulement sur benchmarks choisis

Structure des coûts (scénario HelvetiCart)

Poste	Calcul	\$/mois
Compute Serverless	8 RPU × 3 h × 22 j × \$0,39/RPU-h	≈ 206
Stockage géré	600 GB × \$0,024/GB-mois	≈ 14
S3 staging + divers	forfait	≈ 10
Total	Serverless Frankfurt, 8 RPU base, 3 h/jour ouvré	≈ 230 \$ (~185 CHF)

Points de vigilance :

- Minimum **60 secondes facturées** à chaque réveil: Les petites requêtes éparées coûtent cher proportionnellement.
- Configurer des **usage limits** (`MAX_QUERY_EXECUTION_TIME` , budget mensuel) dès le premier jour.
- En mode **cluster provisionné (RA3)** : Coût fixe prévisible mais qui est payé même sans activité

Démarrage

Prérequis :

- Compte AWS avec droits IAM (`AmazonRedshiftFullAccess` , `AmazonS3ReadOnlyAccess`)
- Un bucket S3 pour le staging des données
- Un client SQL compatible PostgreSQL (psql, DBeaver, DataGrip...)

Créer un workspace Serverless :

- AWS Console → Amazon Redshift → *Serverless* → *Create workgroup*
- Choisir la capacité de base (RPU min. 8)
- Associer un namespace (base de données + credentials)
- Configurer les usage limits pour éviter les surprises de facturation

Connexion :

```
psql -h <workgroup>.redshift-serverless.amazonaws.com
-p 5439
-U admin
-d dev
```

Commandes et opérations clés

Chargement massif depuis S3 (COPY)

```
-- Chargement massif depuis S3 (parallèle, < 30 s pour 150 MB)
COPY orders FROM 's3://bucket/data/'
IAM_ROLE 'arn:aws:iam::123:role/RedshiftS3Role'
FORMAT AS CSV GZIP IGNOREHEADER 1;
```

Requêtes analytiques typiques

```
-- Requête analytique : CA mensuel
SELECT DATE_TRUNC('month', order_date) AS month, SUM(amount) AS revenue
FROM orders GROUP BY 1 ORDER BY 1;

-- Top 5 catégories par canton
SELECT canton, category, SUM(amount) AS revenue FROM orders JOIN products USING (product_id) GROUP BY canton, category QUALIFY
ROW_NUMBER() OVER (PARTITION BY canton ORDER BY revenue DESC) <= 5;
```

Exporter les données (sortie du lock-in)

```
-- Export Parquet vers S3 (anti lock-in)
UNLOAD ('SELECT * FROM orders') TO 's3://bucket/export/orders_'
IAM_ROLE 'arn:aws:iam::123:role/RedshiftS3Role' FORMAT PARQUET;
```

Tuning (mode cluster provisionné uniquement)

```
-- Tuning provisionné : distribution + tri
CREATE TABLE orders (order_id BIGINT, customer_id BIGINT, order_date DATE, amount DECIMAL(10,2))
DISTKEY(customer_id) SORTKEY(order_date);

VACUUM orders; -- récupère l'espace des lignes supprimées
ANALYZE orders; -- met à jour les stats pour l'optimiseur
```

Lire les données S3 sans charger (Spectrum)

```
-- Lire S3 sans charger (Spectrum / data lake)
SELECT * FROM spectrum.external_table LIMIT 100;
```

Lock-in

Dimension	Risque	Détail
Données	Faible	<code>UNLOAD</code> → Parquet ouvert sur S3
SQL	Moyen	PostgreSQL, mais <code>COPY</code> , <code>DISTKEY/SORTKEY</code> , Spectrum sont propriétaires
Pipelines & IAM	Fort	Ingestion, rôles, monitoring, BI : tout tissé dans AWS

Mitigation : garder les données brutes en **Parquet sur S3** (pattern lakehouse) → Redshift devient un moteur remplaçable.

Quand l'utiliser et quand l'éviter

Utilisez Redshift si...	Évitez-le si...
Analytique sur > 100 GB, agrégations lourdes	< quelques dizaines de GB : PostgreSQL suffit
Charge BI / batch intermittente → Serverless	Besoin OLTP ou latence < 100 ms
Déjà sur AWS (S3, IAM, Aurora)	Stratégie multi-cloud ou aversion au lock-in
Équipe SQL, sans envie d'administrer un cluster	Personne pour surveiller les coûts

Références

- **Documentation & pricing** — aws.amazon.com/redshift
- **Billing Serverless** — docs.aws.amazon.com/redshift/latest/mgmt/serverless-billing.html
- **Paper SIGMOD'22** — Armenatzoglou et al., *Amazon Redshift Re-invented* (architecture en détail)
- **Comparatifs critiques** — CloudZero *Redshift Pricing Guide* · Striim *Cloud DW Comparison*